



Persistent

Feature Modules and Lazy Loading

Persistent Interactive | Persistent University



Key Learning Points

- How to create Feature Modules using Angular CLI
- Root Module vs Feature Module
- Eager Loading Feature Modules
- Lazy Loading Feature Modules

Angular Modules

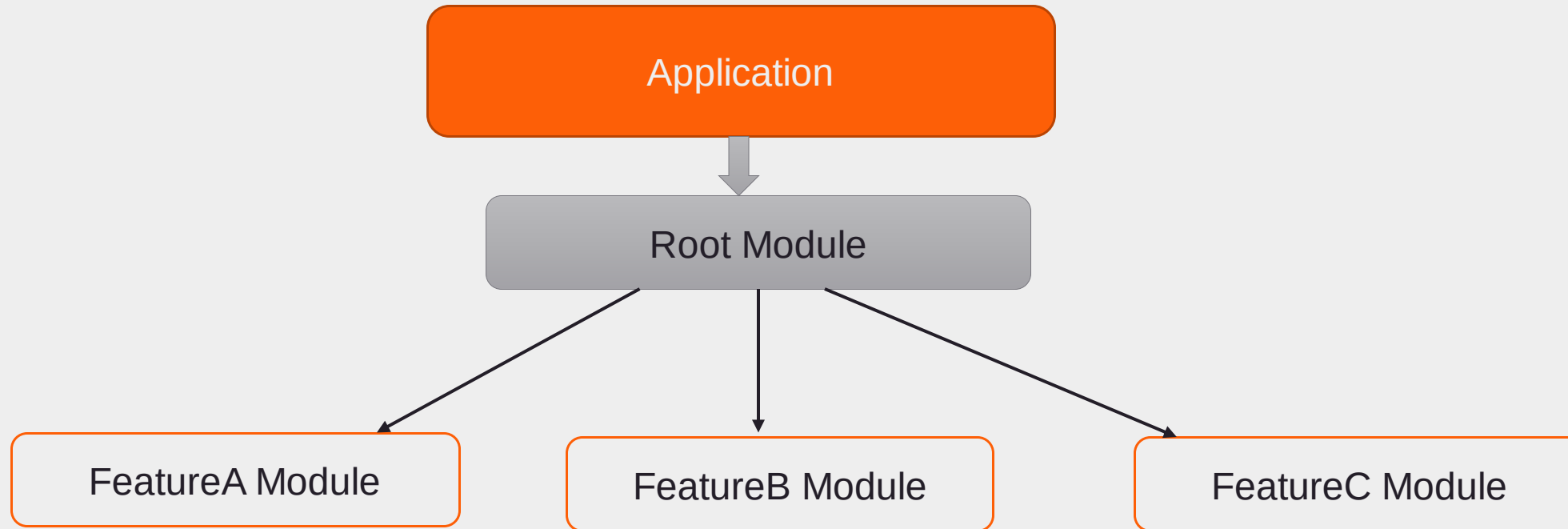
- Modules in Angular consolidate components, directives, and pipes into cohesive blocks of functionality, each focused on a
 - feature area,
 - application business domain,
 - workflow,
 - common collection of utilities.
- Some inbuilt modules in Angular are:
 1. CommonModule
 2. FormsModule
 3. HttpClientModule
 4. RouterModule

Root Module

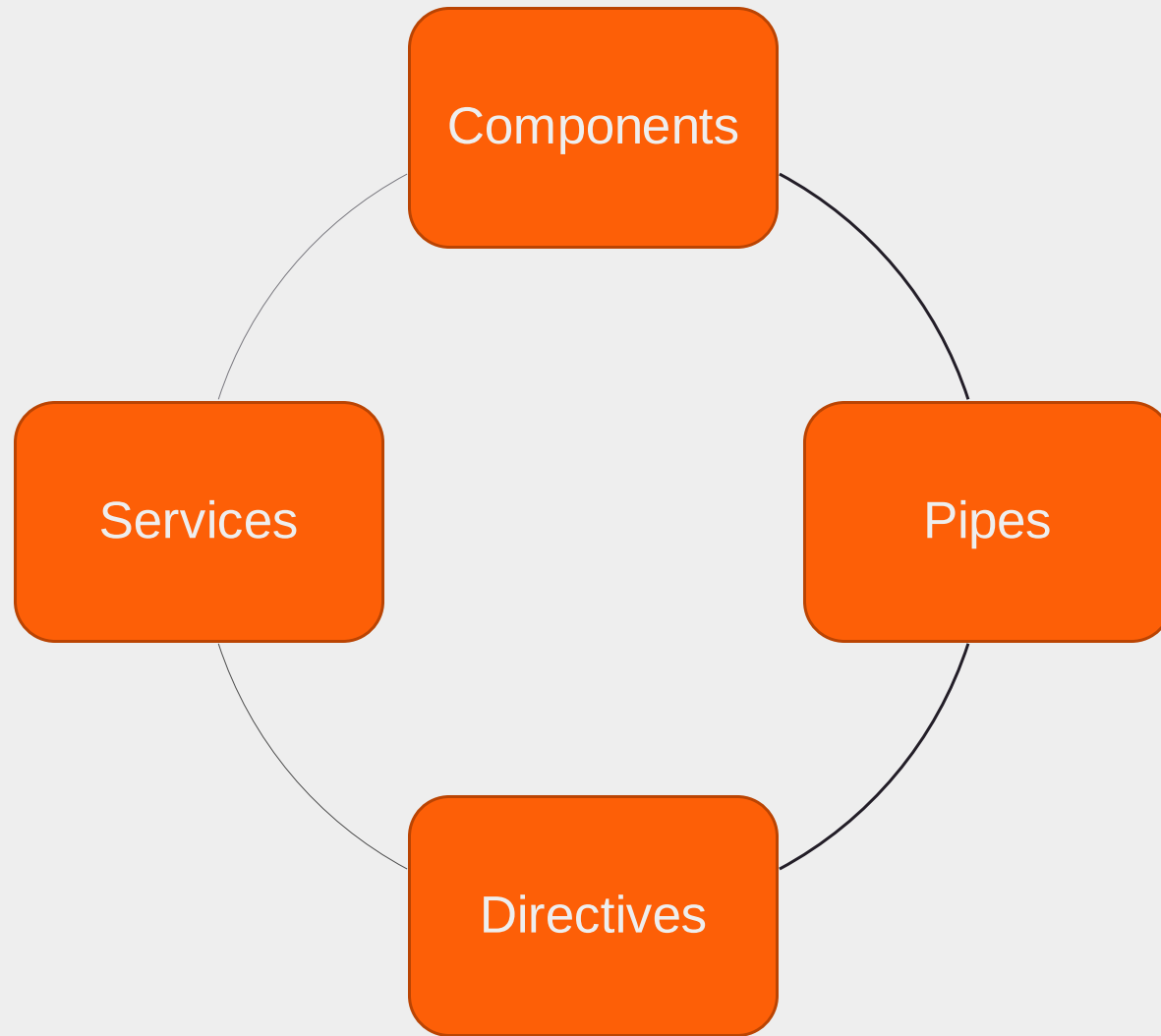
- The Root module popularly named as **AppModule** is bootstrapped to launch the application in the browser.
- Every angular application must have at least one module i.e. the **Root Module**.
- The default project structure provided by Angular CLI provides AppModule in the file **app.module.ts**.
- It is possible to have all components, directives, pipes and services defined in the root module, however splitting them into feature modules helps to achieve high cohesion.

Feature Modules

- We can consolidate components, directives, and pipes into cohesive blocks of functionality, each focused on a feature area, application business domain, workflow, or common collection of utilities.
- With feature modules, we can keep code related to a specific functionality or feature separate from other code.
- Eg: CustomerModule, AuthenticationModule, ShopModule



Feature Modules



Root Module



- ☐ One per application
- ☐ Launches the application
- ☐ Imports other feature modules
- ☐ Has bootstrap property

Feature Modules



- ☐ Many in an application
- ☐ Based on feature/functionality
- ☐ Can import only required modules
- ☐ Does not have bootstrap property

Generate a Feature Module

ng generate module Customer

- This will generate a folder named “Customer” with a module file “**customer.module.ts**”
- Note that the @NgModule decorator’s **imports array** already has CommonModule added to it.
- This is required for the core features of Angular like *ngIf, *ngFor and other annotations.
- Any other additional modules to be used by the feature module can be added to this array.

Importing the CommonModule

- Every feature module has to import the CommonModule.
- CommonModule only contains information for common directives such as `ngIf` and `ngFor` which are needed in most templates.
- The BrowserModule configures the Angular app for the browser which needs to be done only once. Hence this is imported only in the RootModule.

Add a Component to module

- Add a component to the Customer Module with below command.

ng generate component Customer/Dashboard

- Notice that the generated component DashboardComponent is automatically added to the **declarations** array.
- Similarly when new directives and pipes are created in the module, they are automatically added to the declarations array.

Adding a Routing Module

- The general practice is to create a separate route file which has all routing configuration at the application root level.
- Eg: app.routing.ts

```
import { RouterModule, Routes } from '@angular/router';  
  
export const routes: Routes = [  
  { path: 'component-one', component: ComponentOne },  
  { path: 'component-two', component: ComponentTwo }  
];
```

Import routes and Configure with RouterModule

- The routes array is in turn is imported into the app.module.ts

```
import { routes } from './app.routing';

@NgModule({
  imports: [
    BrowserModule, RouterModule.forRoot(routes)
  ],
  declarations: [
    AppComponent, ComponentOne, ComponentTwo
  ],
  bootstrap: [ AppComponent ]
})
export class AppModule {
}
```

Notice the use of `forRoot()` to configure the application routes

Handling routes in Feature Modules

- When a new feature module is created, it will have its own route file to define routes referred to as **child routes**.
- Angular CLI command to generate this file:
- **ng module Customer --routing**
- This will generate 2 files:
 - customer.module.ts
 - customer.routing.ts

customer.routing.ts

- The file **customer.routing.ts** is used to define child routes.

```
import { RouterModule, Routes } from '@angular/router';  
export const routes: Routes = [  
  { path: 'add', component: CustomerComponent },  
  { path: 'reports', component: CustReportsComponent }  
];
```

Import the routing file

- Customer.module.ts

```
import { routes } from './customer.routing';

@NgModule({
  imports: [
    CommonModule, RouterModule.forChild(routes)
  ],
  declarations: [
    ComponentOne, ComponentTwo
  ] })
export class CustomerModule {
}
```

Notice the use of `forChild()` to configure the child routes

forRoot() vs forChild()

- Used to configure routes in the Root module i.e AppModule
- Used one time- one per entire application
- Has Injector configuration for services like Router along with directives like routerLink and router-outlet.

- Used to configure child routes in the Feature Modules.
- Used multiple times- in every feature modules which has routes configuration.
- Only has directives like routerLink and router-outlet.

Importing a Feature Module into the Root Module

- Two ways of importing feature modules into the Root Module

1. Eager Loading

2. Lazy Loading

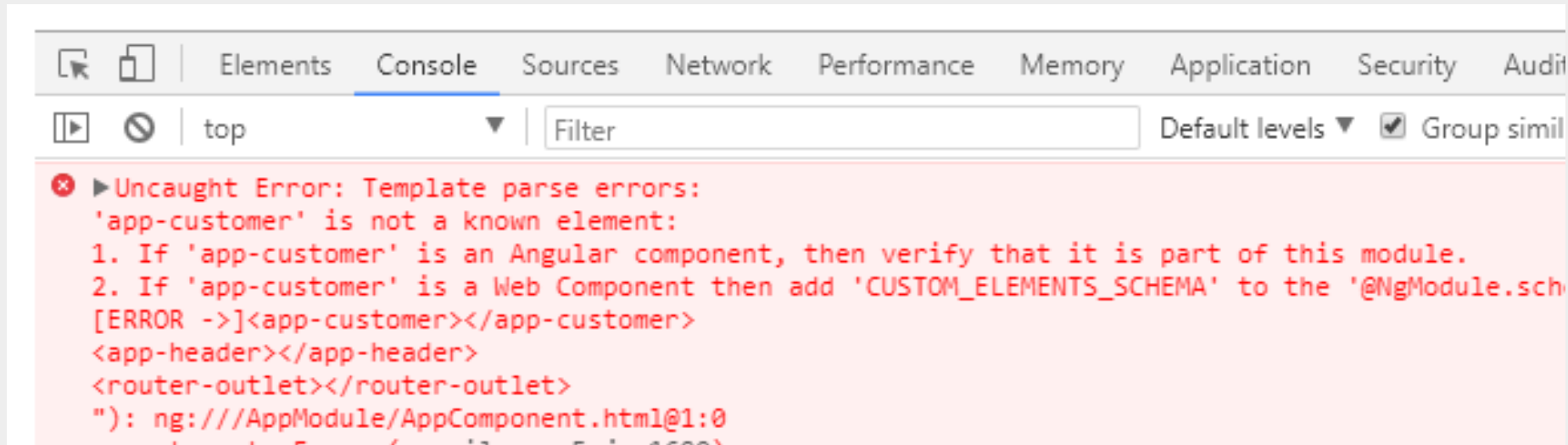
Eager Loading

- This approach refers to loading a feature module right at the time of bootstrapping i.e when the Root module is being loaded.
- Add the feature module to the imports array.

```
@NgModule({  
  declarations:[],  
  imports: [  
    BrowserModule,FormsModule,CustomerModule  
  ],  
  providers:[]  
  
})  
export class AppModule { }
```

Visibility of components in a module

- By default, components defined in an NgModule, is not visible outside to other modules.
- Eg: In the Customer Module, if there is a CustomerDashboard Component then, although the module is added to the imports array, the component cannot be directly used.
- Adding the directive **<customer-dashboard></customer-dashboard>** in **AppComponent** will throw an error.



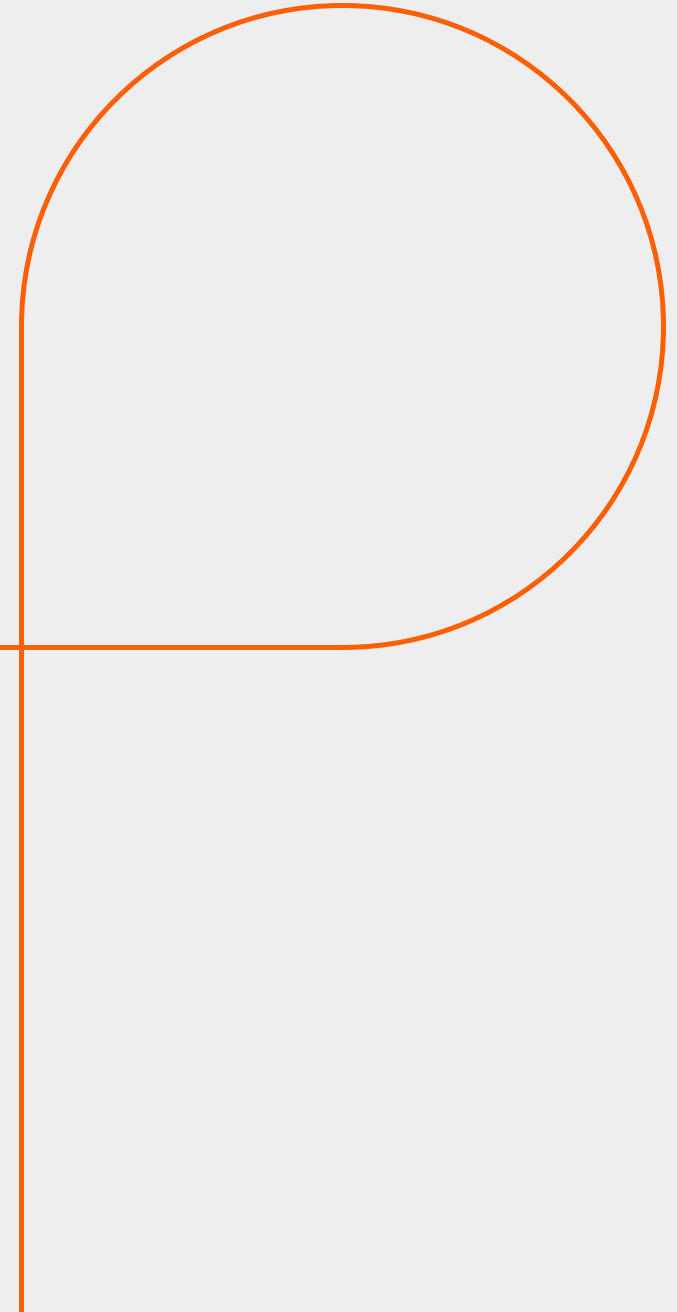
Exports Array

- To resolve the error, add the CustomerComponent to the exports array in the **customer.module.ts** file.

```
@NgModule({  
  imports: [  
    CommonModule  
  ],  
  declarations: [CustomerComponent],  
  exports:[CustomerComponent]  
})  
export class CustomerModule { }
```

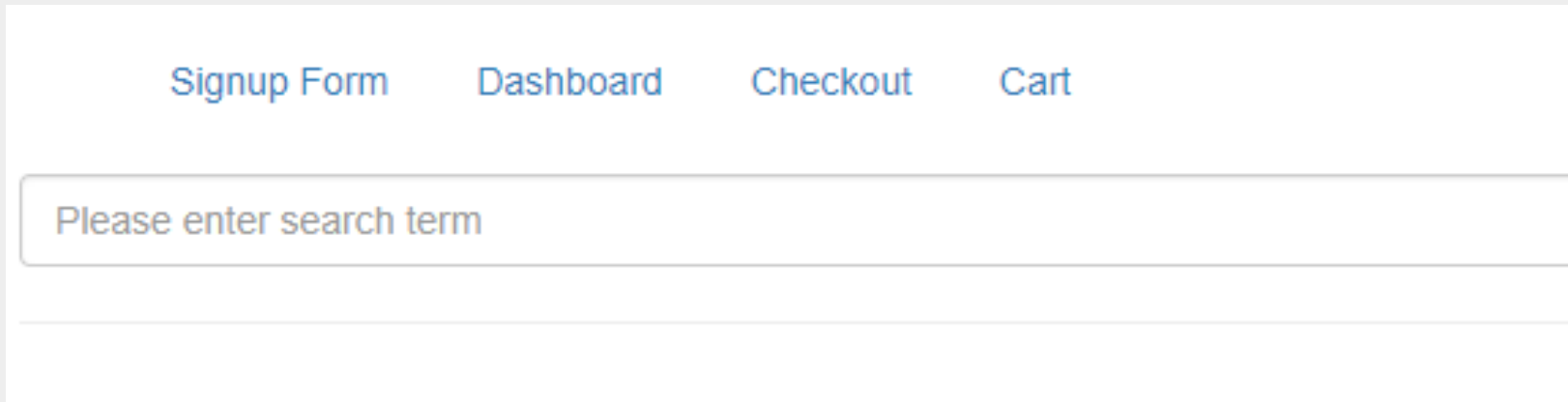
Lazy Loading of Modules

Persistent University



Lazy Loading Modules

- With lazy loading feature, it's possible to have features loaded only when the user navigates to their routes for the first time.
- This improves the application performance along with reducing the bundle size.



- Here, Lets say we want to load only two components- Signup Form and Dashboard when app is loaded.
- The Checkout and Cart components can be loaded only when user navigates to them. This is referred to as **Lazy loading**.

Implementation of Lazy Loading

- Create the basic setup of application routing by creating a menu and configuring the routes.

```
let routes:Routes = [  
  {path:'',component:SignUpComponent},  
  {path:'dashboard',component:CustomerDashboardComponent},  
  {path:'form',component:SignUpFormComponent},  
];
```

```
imports: [  
  BrowserModule,ReactiveFormsModule,FormsModule,CustomerDashboardModule,  
  RouterModule.forRoot(routes)  
]
```

CustomerDashboardModule is
loaded during bootstrapping

ShopModule

- Then create a feature module named “shop” for all shopping related components like checkout, order, cart etc.
- Create appropriate components in the “shopModule”.
- The ShopModule will have its own routing file to handle its roots. Lets say it is **shop-routing.ts** file.

```
export const routes: Routes = [  
  { path: 'cart', component: CartComponent },  
  { path: 'checkout', component: CheckoutComponent },  
  { path: 'confirm', component: ConfirmComponent },  
];
```

shop.module.ts file

- Now the main module file will configure the routing by importing the routes from shop-routing.ts file (previous slide).

```
import { routes } from './shop.routing';
@NgModule({
  imports: [
    CommonModule,RouterModule.forChild(routes)],
  declarations: [CartComponent,CheckoutComponent,ConfirmComponent],
})
export class ShopModule {
}
```

- Note the use of **RouterModule.forChild()** since these are child routes.
- All Feature Modules must configure their routes by using this method.

Syntax for Lazy Load

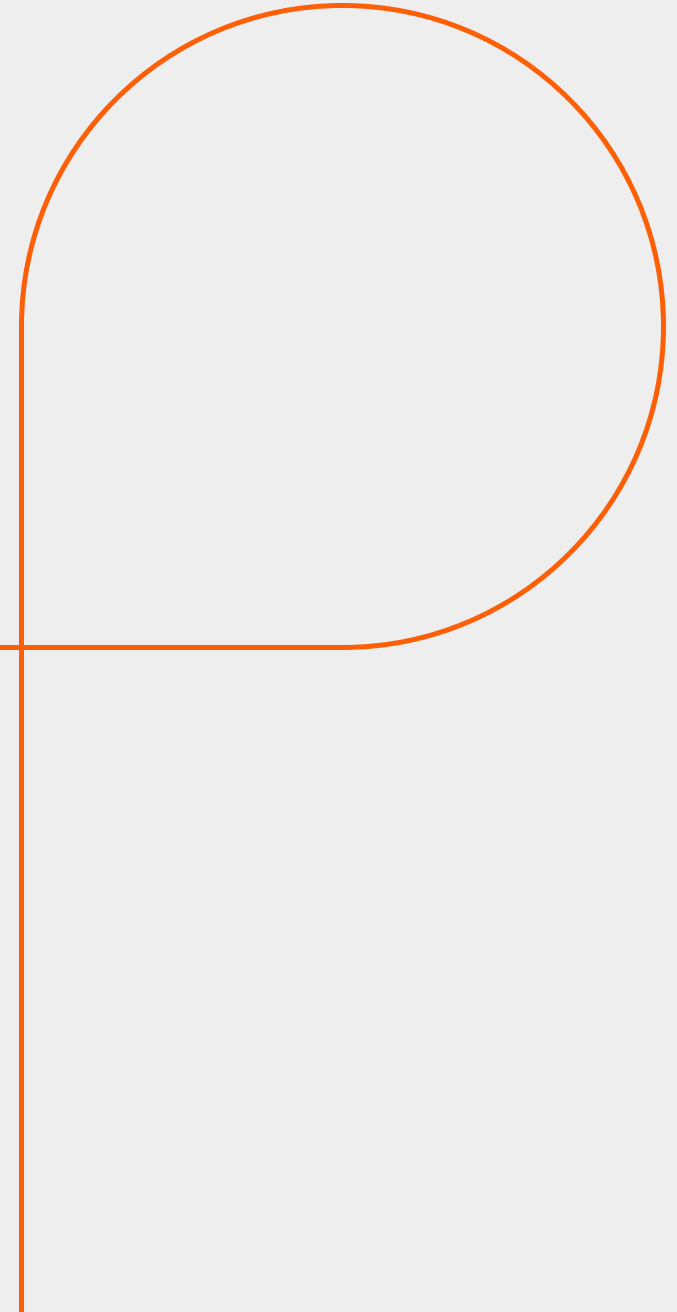
- Use the property `loadChildren` to load all child routes pertaining to shop module.

```
let routes:Routes = [  
  {path:'',component:SignUpComponent},  
  {path:'dashboard',component:CustomerDashboardComponent},  
  {path:'form',component:SignUpFormComponent},  
  {path:'shop',loadChildren:'../shop/shop.module#ShopModule'}  
];
```

- Note the syntax for lazy load. The complete path of the module needs to be specified here.
- **Remember not to add ShopModule to the imports array in AppModule.**

How Providers behave when modules are imported?

Persistent University



Providers of AppModule

- The providers in AppModule are visible to the entire application.
- This makes the providers visible to every class in the application that knows the provider's lookup token, or knows its name.
- Eg: When HttpClientModule is imported by a module, the service 'Http' is available across the application.

Visibility of Services

- When you import an NgModule, Angular adds the contents of its providers list to the application root module's provider list.

```
@NgModule({  
  declarations: [AppComponent],  
  imports: [  
    BrowserModule,  
    CustomerModule,  
  ],  
  providers: [CustomerService],  
  bootstrap: [AppComponent]  
})  
export class AppModule { }
```



```
@NgModule({  
  declarations:  
    [DashboardComponent],  
  imports: [ CommonModule ],  
  exports: [],  
  providers: [CustomerService],  
})  
export class CustomerModule { }
```

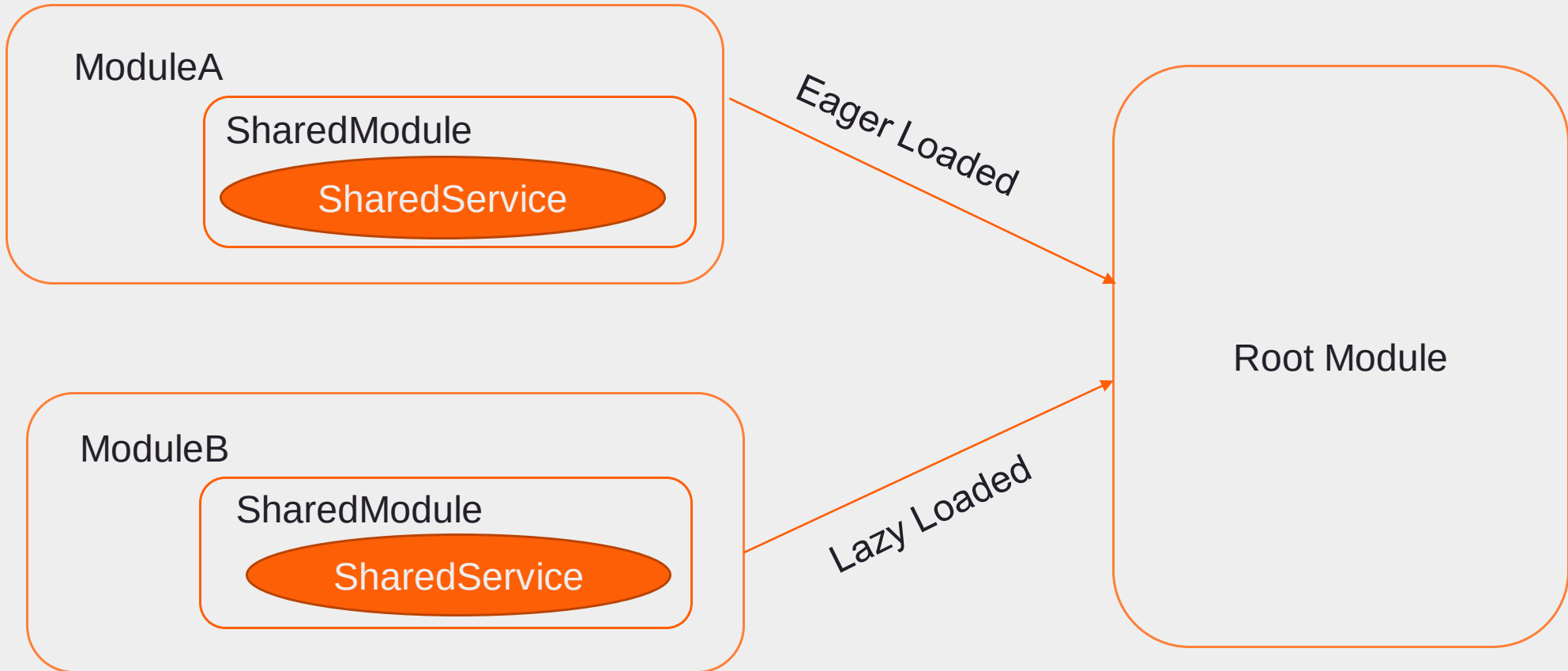


- When CustomerModule is imported into the AppModule, the CustomerService automatically gets added to AppModule's list of providers.

SharedModule

- Sometimes there is a need for a Shared Module that contain common directives, services or pipes which are used by all feature modules.
- An application may end up loading a Shared Module more than once which means it ends up providing a service more than once.
- But ideally services are supposed to be singletons.
- This scenario occurs when AppModule loads ModuleA eagerly and ModuleB is lazily loaded and both these modules in turn are loading SharedModule.

Shared Service provided twice



ModuleWithProviders

- The Shared Service is being provided in a different way in the shared Module.

```
import { NgModule, ModuleWithProviders } from
'@angular/core';
export class SharedModule {
  static forRoot(): ModuleWithProviders {
    return {
      ngModule: SharedModule,
      providers: [ SomeService ]
    };
  }
}
```

Best practice while exporting services

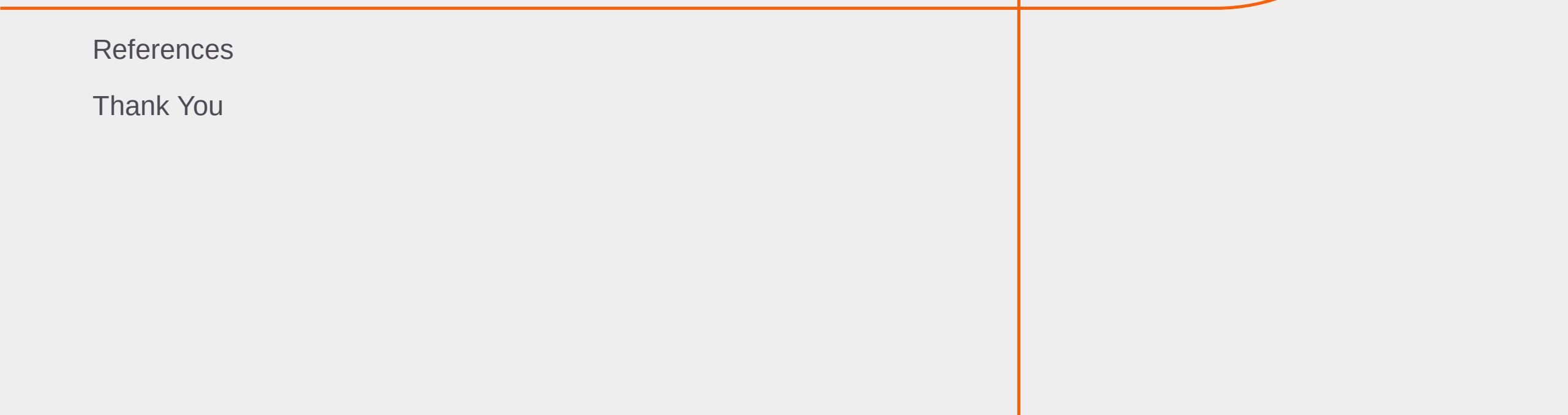
- As a rule of thumb, always use the forRoot syntax when exporting services from feature modules, unless you have a very special need that requires multiple instances at different levels of the dependency injection tree.

Summary: Session

With this we have come to an end of our session, where we discussed about

- How to create Feature Modules using Angular CLI
- Root Module vs Feature Module
- Eager Loading Feature Modules
- Lazy Loading Feature Modules

Appendix



References

Thank You

References

- <https://alligator.io/angular/lazy-loading/>
- <https://angular.io/guide/feature-modules>
- <https://alligator.io/angular/providers-shared-modules/>

Key Contacts :

Persistent University :

- Archana Ghatigar
archana_ghatigar@persistent.co.in



Thank you!

Persistent Interactive | Persistent University

